Intelligent Electric Motor Health Monitoring

Using Vibration Analysis and Machine Learning

Project Proposal

Reference: MathWorks MATLAB/Simulink Challenge Project 253 — Fault Detection for Electric Motors Using Vibration Analysis, developed in collaboration with STMicroelectronics.

1. Project Overview

Electric motors are fundamental components in industrial systems, where unexpected failures can cause production downtime, equipment damage, and significant maintenance costs. Traditional maintenance is generally either reactive or preventive, and neither necessarily reflects the actual condition of the machine.

Predictive maintenance instead uses measurements and data-driven methods to identify abnormal behavior and developing faults. Vibration is particularly useful because mechanical abnormalities such as bearing defects, rotor imbalance, and shaft misalignment can produce characteristic changes in vibration signals.

Proposed system: an AI-based motor health monitoring pipeline that analyzes vibration data to diagnose known faults, detect abnormal behavior, monitor condition evolution, forecast selected condition indicators, and demonstrate deployment as a usable application.

2. Problem Statement

Electric motors can develop mechanical and electrical faults gradually before catastrophic failure. Examples include bearing faults, rotor imbalance, shaft misalignment, electrical faults, excessive friction, and other abnormal operating conditions.

> **Central question:** How can vibration measurements be transformed into useful information about the current and future health of an electric motor?

3. Proposed Solution

The proposed system follows a complete data-to-decision pipeline:

Stage	Main Output
Data acquisition	Public vibration dataset and/or simulated vibration data
Signal processing	Cleaned and segmented vibration windows; spectral representations
Feature extraction	Time-domain and frequency-domain features
Fault diagnosis	Known fault class + confidence
Anomaly detection	Normal/abnormal decision + anomaly score
Condition monitoring	Health/condition indicator + trend
Forecasting	ARIMA-based condition forecast
Deployment	Python application/dashboard; embedded deployment optional

4. Project Objectives

- Understand healthy and faulty motor vibration characteristics.
- Develop a reproducible preprocessing and segmentation pipeline.
- Extract meaningful time-domain and frequency-domain features.

- Develop and compare machine-learning models for fault diagnosis.
- Investigate anomaly detection for abnormal behavior not explicitly represented during training.
- Develop a vibration-derived condition indicator and study its evolution over time.
- Apply ARIMA to investigate short-term forecasting of selected condition indicators.
- Package the trained ML pipeline into a usable application/API.

5. Signal Processing and Data Analysis

Raw vibration data will be investigated in the time, frequency, and, where useful, time-frequency domains. Analysis will include waveform visualization, sampling-rate checks, filtering where justified, FFT/PSD analysis, and STFT/spectrograms.

Time-domain features: RMS, variance, standard deviation, peak, peak-to-peak amplitude, kurtosis, skewness, and crest factor.

Frequency-domain features: dominant frequency, spectral energy, band power, spectral centroid, spectral entropy, and harmonic information.

6. Feature Engineering and Windowing

Continuous recordings will be divided into appropriate windows. Each window becomes a candidate ML sample from which features are calculated and associated with the relevant operating condition or fault label.

Key methodological requirement: train/test splitting must prevent leakage between highly correlated neighboring windows, particularly when multiple windows originate from the same recording or operating run.

7. Fault Diagnosis

Where supported by the dataset, the supervised-learning problem will be formulated as multiclass classification. Candidate classes may include healthy operation, bearing fault, rotor imbalance, shaft misalignment, and electrical fault. Final classes depend on the selected dataset.

The desired output is a predicted condition together with a confidence/probability rather than only a class label.

8. Anomaly Detection

Fault classification and anomaly detection will be treated as different tasks. A classifier asks: “Which known condition is this?”, while an anomaly detector asks: “Does this behavior look unlike normal operation?”

Candidate methods include Isolation Forest, One-Class SVM, and an autoencoder.

9. Motor Health Indicator

The project will investigate whether vibration-derived features can form useful condition indicators whose values change meaningfully as motor condition changes. The resulting indicator will be analyzed as a time series rather than treated as an arbitrary score.

10. ARIMA Forecasting

ARIMA will be investigated for time-series modeling of a selected condition indicator. The workflow will include stationarity analysis, ACF/PACF inspection, differencing where appropriate, model selection, fitting, residual analysis, and forecasting.

The project will avoid overclaiming: ARIMA will be evaluated as a forecasting tool for condition indicators, not assumed to predict an exact motor-failure time unless the dataset genuinely supports such a conclusion.

11. Machine-Learning Experiments

Model	Role
Logistic Regression	Baseline
SVM	Nonlinear classification baseline
Random Forest	Robust tree-based model
LightGBM	High-performance gradient boosting
1D CNN	Optional deep-learning comparison
CNN + spectrogram	Optional time-frequency representation learning

12. Explainability

Feature importance and, where appropriate, SHAP analysis will be used to investigate which vibration characteristics influence model predictions, connecting ML behavior with physically meaningful signal characteristics.

13. Deployment

The project will move beyond a notebook by packaging the trained pipeline into a usable application. A practical initial deployment target is a Python inference service using FastAPI, combined with a Streamlit dashboard.

The dashboard can display raw vibration, FFT/spectrogram views, predicted fault, confidence, anomaly score, condition trend, and ARIMA forecast.

14. Optional Embedded Deployment

The original MathWorks project proposes real-time deployment on an STM32 NUCLEO-H743ZI2 with accelerometer acquisition. Because the project window is only 1–3 weeks and hardware is not guaranteed, embedded deployment will be treated as an optional extension rather than a core requirement.

15. Data Strategy

Primary: select a public motor-vibration dataset containing raw vibration signals, sampling information, fault labels, and preferably multiple operating conditions or severities.

Secondary: if useful, generate controlled synthetic vibration data using MATLAB/Simulink.

Optional: if the software pipeline is completed early, collect a small real-world demonstration using a motor and accelerometer.

16. Experimental Questions

- Can vibration-derived features reliably distinguish healthy and faulty motor conditions?
- Which time-domain and frequency-domain features contribute most strongly to diagnosis?
- How do classical ML models compare with deep-learning approaches for the selected dataset?
- Can anomaly detection identify abnormal behavior without training on every possible fault?
- Can a vibration-derived condition indicator provide a meaningful representation of motor-health evolution?
- Can ARIMA provide useful short-term forecasts of that condition indicator?
- Can the final ML pipeline be transformed from a research notebook into a deployable application?

17. Evaluation Strategy

Classification: accuracy, precision, recall, F1-score, confusion matrix, and suitable ROC/PR analysis.

Anomaly detection: precision/recall, false-alarm behavior, detection rate, and appropriate threshold-based evaluation.

Forecasting: MAE, RMSE, forecast error, and residual analysis.

Deployment: inference latency, model size, reproducibility, and prediction reliability.

18. Expected Final Product

- Vibration-data analysis and preprocessing pipeline.
- Time- and frequency-domain feature-extraction pipeline.
- Validated multiclass fault-diagnosis model.
- Anomaly-detection component.
- Condition/health monitoring component.
- ARIMA forecasting component.
- Explainability/feature-importance analysis.
- Deployable Python application/dashboard.
- Technical presentation and project documentation.

19. Scope and Priorities

Priority	Deliverables
Must have	Dataset, EDA, signal processing, windowing, feature extraction, baseline + best ML model, evaluation, fault diagnosis, ba
Should have	Anomaly detection, condition indicator, ARIMA forecasting, explainability, model comparison, error analysis
If time allows	CNN, spectrogram CNN, FastAPI, Docker, real-time simulation, hardware, STM32/edge deployment

20. Proposed 1–3 Week Timeline

Period	Focus
Days 1–2	Dataset selection, domain understanding, EDA, sampling-rate and signal inspection
Days 3–4	Preprocessing, segmentation, FFT/PSD, feature extraction
Days 5–7	Baseline ML, SVM/Random Forest/LightGBM, evaluation and leakage checks
Days 8–9	Anomaly detection
Days 10–11	Condition indicator and temporal analysis
Days 12–13	ARIMA modeling and forecasting
Day 14	Explainability and error analysis
Days 15–18	Model packaging, Streamlit dashboard, optional FastAPI
Remaining	Optional CNN/hardware/edge work, documentation, presentation and demo

21. Risks and Mitigation

Risk	Mitigation
Dataset does not support ARIMA	Treat forecasting as an extension and choose an appropriate condition indicator/time-series structure.
Dataset is too small	Use careful windowing, justified augmentation, and/or controlled synthetic data.
Synthetic-to-real gap	Document the limitation clearly and treat real-world validation as future work.

Risk	Mitigation
Deep learning provides no improvement	Use the result as a legitimate comparison of representation-learning complexity versus engineered features
Hardware consumes too much time	Keep hardware optional; the software pipeline remains the core deliverable.
Scope explosion	Protect the classification pipeline first; add advanced components only after the core system works.

22. Technology Stack

Python: NumPy, Pandas, SciPy, Matplotlib, Seaborn, scikit-learn, LightGBM, and potentially PyTorch.

Time series: statsmodels.

Deployment: FastAPI, Streamlit, joblib and/or ONNX where appropriate.

Optional: MATLAB/Simulink for controlled motor/fault simulation and possible embedded workflow.

23. Why This Project Fits an AI/ML Data Analysis Internship

The project covers the complete data-to-decision pipeline: raw data → analysis → preprocessing → feature engineering → machine learning → evaluation → interpretation → deployment.

It also provides room to investigate supervised learning, anomaly detection, time-series forecasting, and deep learning without requiring every component to be completed for the core project to succeed.

The vibration domain also allows engineering knowledge to contribute directly to the ML process. Sampling, spectral content, harmonics, and statistical vibration indicators can be connected to model behavior and physical motor faults.

24. Final Proposal Statement

Proposed project: Intelligent Electric Motor Health Monitoring Using Vibration Analysis and Machine Learning

Objective: Develop a data-driven system that analyzes electric-motor vibration signals to diagnose known faults, detect previously unseen abnormal behavior, monitor motor-condition evolution, forecast selected condition trends using ARIMA, and demonstrate deployment of the resulting ML pipeline as a practical monitoring application.

Core technologies: Python • Signal Processing • Machine Learning • Anomaly Detection • Time-Series Analysis • ARIMA • Data Visualization • ML Deployment